

UCLouvain

LINFO1121 2026-2027

Algorithmique et Structure de données

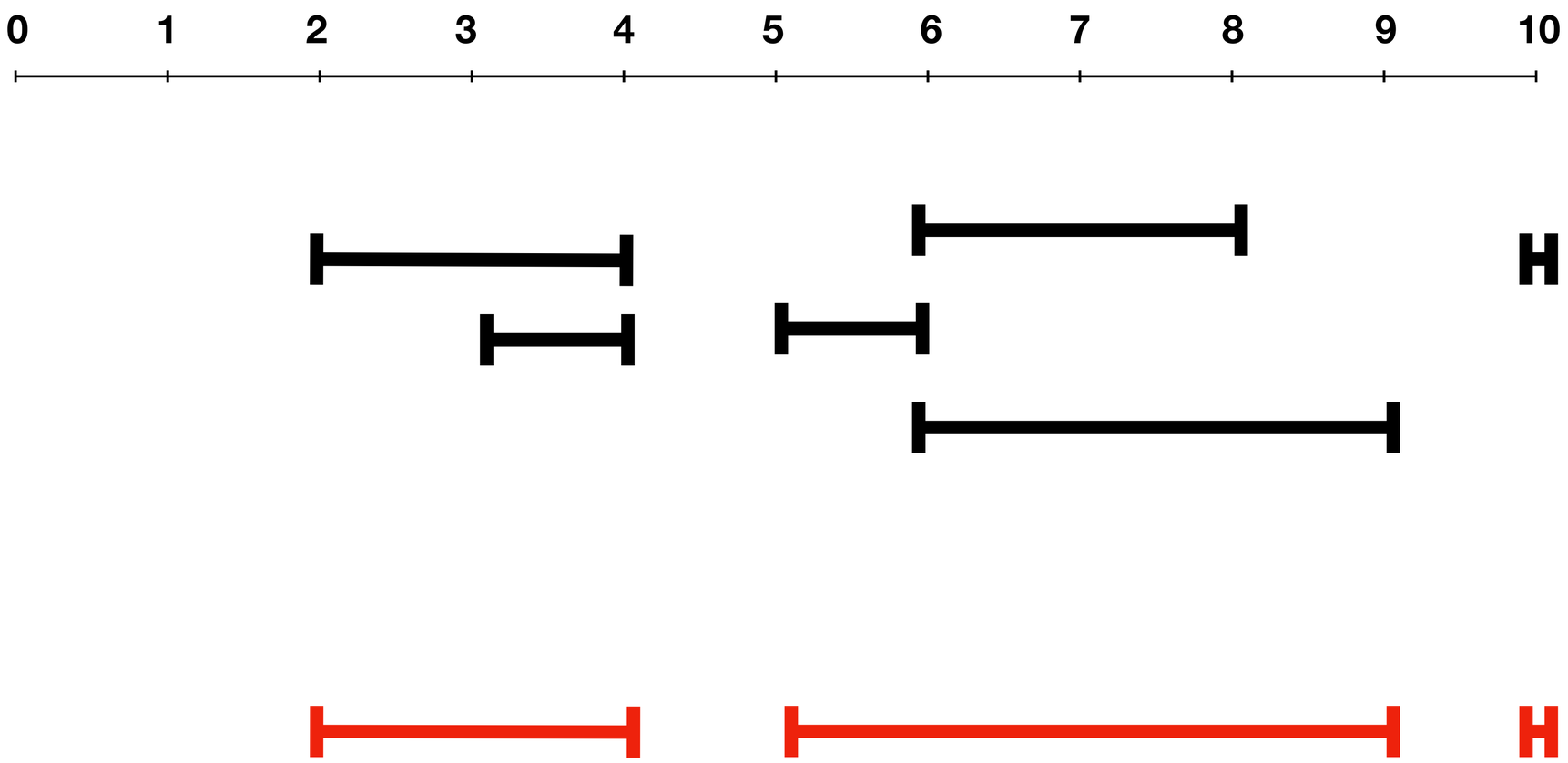
Restructuration 2: Sorting

Prof pierre.schaus@uclouvain.be

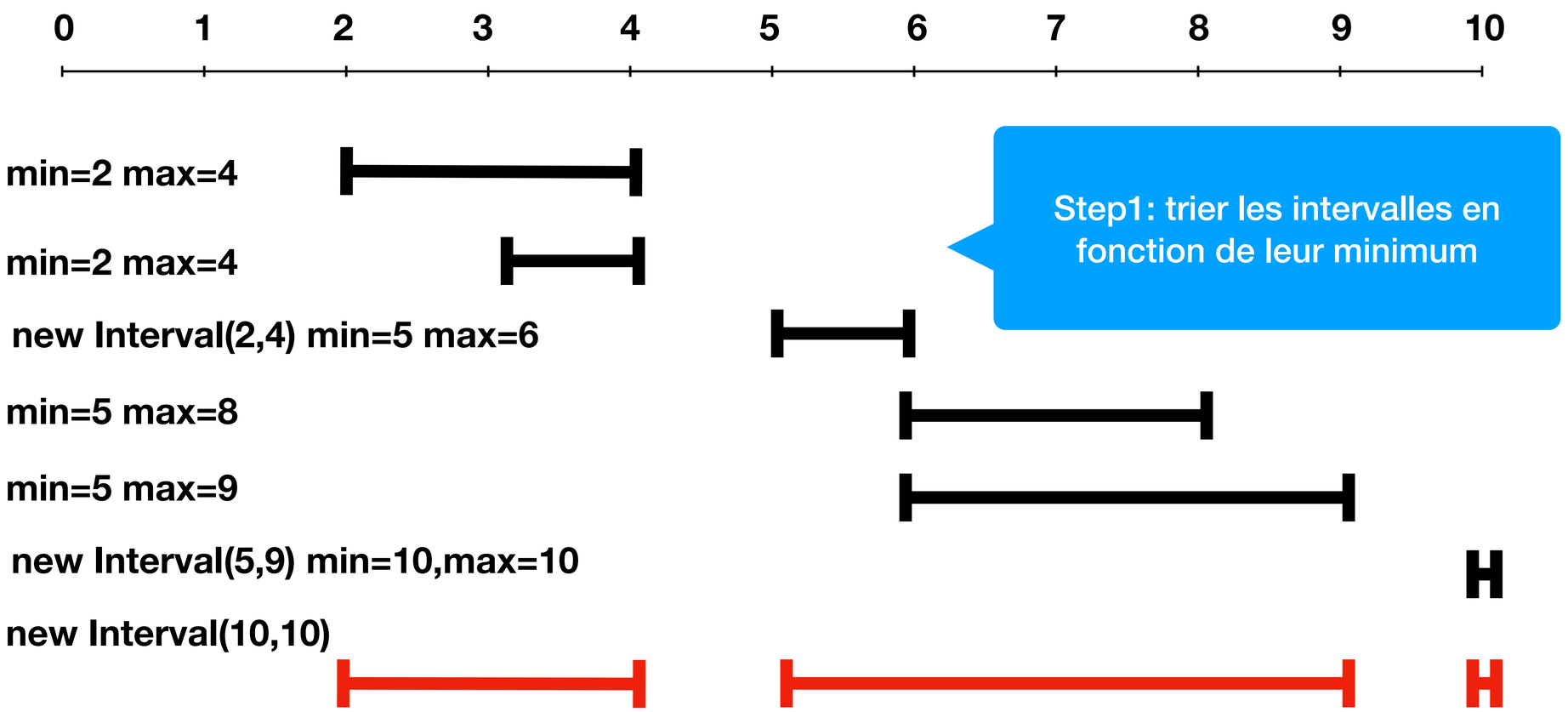
Union of Intervals

```
public static Interval[] union(Interval[] intervals) {
    // TODO
    return new Interval[]{};
}
```

Union of intervals: Example



Algo1: two steps



Algo1

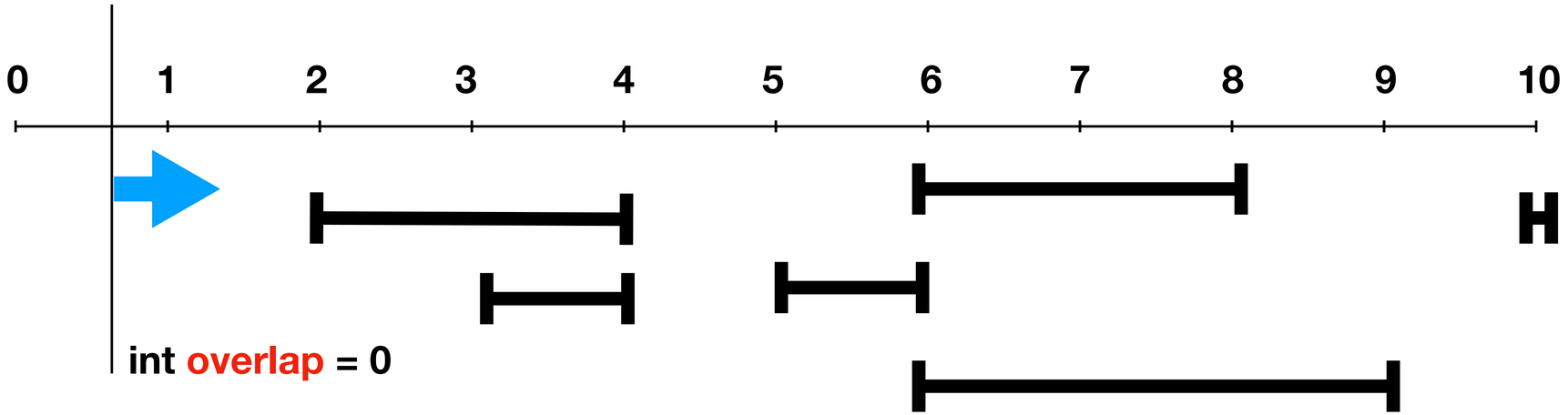
```
public static Interval [] union(Interval [] intervals) {
    if (intervals.length == 0) return new Interval[0];
    ArrayList<Interval> res = new ArrayList<>();
    Arrays.sort(intervals);
    int min = intervals[0].min;
    int max = intervals[0].max;
    int i = 1;
    while (i < intervals.length) {
        if (intervals[i].min > max) {
            res.add(new Interval(min,max));
            min = intervals[i].min;
            max = intervals[i].max;
        } else {
            max = Math.max(max,intervals[i].max);
        }
        i++;
    }
    res.add(new Interval(min,max));
    return res.toArray(new Interval[0]);
}
```

! We change the input (sort), not always considered a good practice

Time complexity ?

- O(n) where n = number of input intervals
- Θ(n) where n = number of input intervals
- O(n log(n)) where n = number of input intervals
- Θ(n log(n)) where n = number of input intervalle

Alternative Approach: Sweep-line Algo (more generic)



- Sequence of events *(time,increment)* at beginning and end of each interval
  - $events = \langle (I.start, +1) \mid \forall I \in Intervals \rangle + \langle (I.end, -1) \mid \forall I \in Intervals \rangle$
- Sort lexicographically with criteria *(time,-increment)*
  - (2,+1),(3,+1),(4,-1),(4,-1),(5,+1),(6,+1),(6,+1)(6,-1),(8,-1),(9,-1),(10,+1),(10,-1)
- Iterate on it and maintain overlap = #intervals that intersect the sweep line
- For each new time step, process events one by one updating the overlap counter
- When all events are processed for a time-step, if overlap=0 then a new interval is detected

Sweep Line Algo: pseudo code

```java
public static List<int[]> computeUnion(int[] min, int[] max) {
    int n = min.length;
    Event[] S = new Event[2 * n + 1];

    for (int i = 0; i < n; i++) {
        S[2 * i] = new Event(min[i], 1);
        S[2 * i + 1] = new Event(max[i] + 1, -1);
    }

    S[2 * n] = new Event(Integer.MAX_VALUE, 0);
    Arrays.sort(S);

    List<int[]> union = new ArrayList<>();
    int start = S[0].x;
    int overlap = S[0].delta;

    int i = 1;
    while (i < 2 * n) {
        overlap += S[i].delta;
        if (overlap == 0) {
            union.add(new int[]{start, S[i].x});
            start = S[i + 1].x;
        }
        i += 1;
    }

    return union;
}
```

"event points" creation

Process of events

Detect a new interval

```java
class Event implements Comparable<Point> {
    int x;
    int delta;

    Event(int x, int delta) {
        this.x = x;
        this.delta = delta;
    }

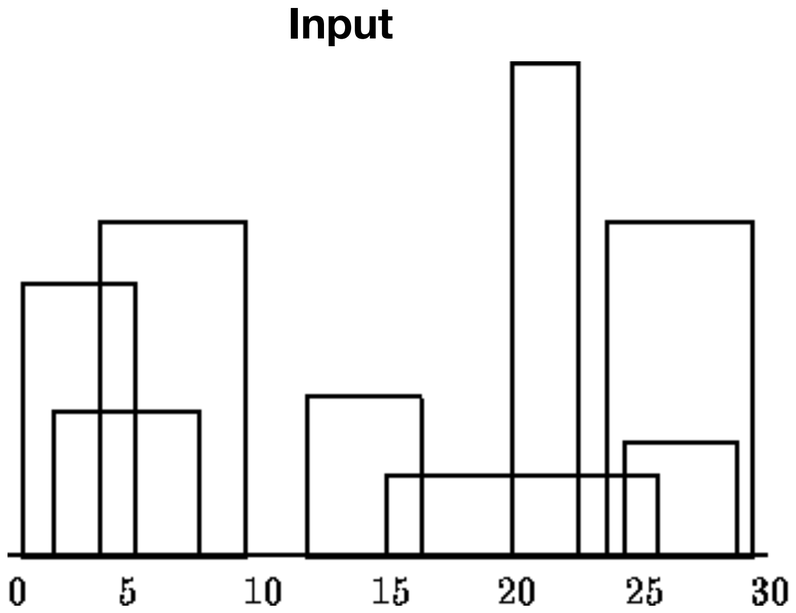
    @Override
    public int compareTo(Event o) {
        if (this.x != o.x) return Integer.compare(this.x, o.x);
        return -Integer.compare(this.delta, o.delta);
    }
}
```

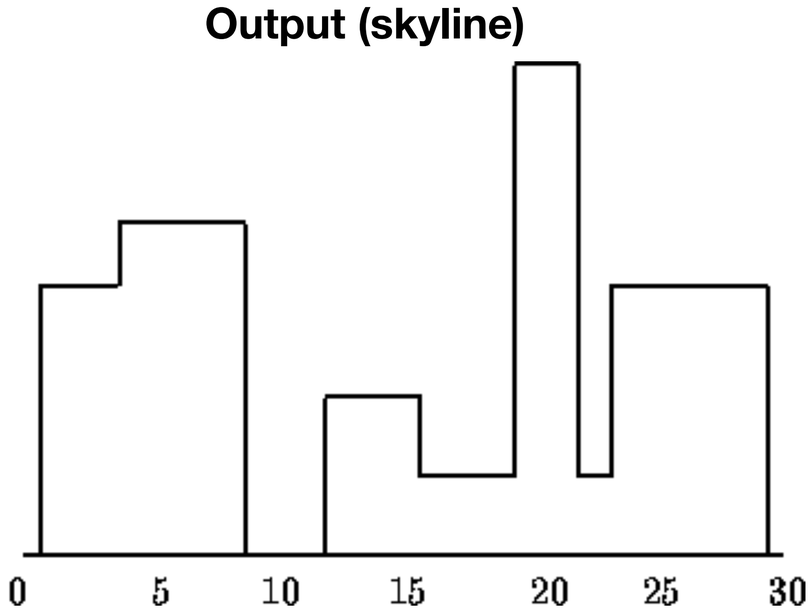
Lexico sort

Sweep-line technique

- Is a techniques that comes from “computational geometry”

- We will see it in another context:
  - Compute the the “skyline” from a set of rectangles





Warmup

- If you have the information that in an array your have 1x0, 3x1, 2x5, can you reconstruct the sorted input array ?
  - For sure!: [0,1,1,1,5,5]

- This is exactly what costing sort algorithms is doing in two steps:

1. Count the values
2. Reconstruct the sorted array based on the counters

Reconstruct the sorted array

- What is the time complexity for reconstructing the sorted array based on the counters: k1 x 0, k2 x 1, k3 x 2 ?
  - O(k1+k2+k3) ?
  - Θ(k1+k2+k3) ?
  - O(n log(n)) où n=k1+k2+k3 ?
  - Θ(n log(n)) où n =k1+k2+k3 ?

Counting sort

```
int [] input =  new int[]{0,1,2,0,0,2,0,1,0,2,1,0,1,1,0,0,2,2,2,2,0};

int [] counters =  new int[3];
for (int i : input) counters[i]++;
int [] ouput = new int[input.length];
int j = 0;
for (int i = 0; i <= 2; i++) {
    while (counters[i] > 0) {
        counters[i]--;
        ouput[j] = i;
        j++;
    }
}
```

Counting sort time complexity {0,1,2}

- n = number of input values
  - $O(n)$
  - $\Theta(n)$
  - $O(n \log(n))$
  - $\Theta(n \log(n))$
  - $O(n^2)$
  - $\Theta(n^2)$

Counting sort time complexity for k values ?

- Must sort values between *0* and *k* ? Time complexity for sorting *n* values ?
  - O(*n*)
  - $\Theta(n)$
  - O(*n*+*k*)
  - $\Theta(n+k)$
  - O(*n* log(*n*))
  - $\Theta(n \log(n))$

The algorithm



```
public static void countingSort(int[] values) {
    int[] buckets = new int[100]; // k = 100
    // Count occurrences of each value in the input array
    for (int i : values) {
        buckets[i]++;
    }
    int idx = 0;
    for (int i = 0; i < 100; i++) {
        for (int j = 0; j < buckets[i]; j++) {
            values[idx] = i;
            idx++;
        }
    }
}
```

More Complex Counting Sort

- Suppose I have pairs $(a_i, b_i) \forall i \in [0..n-1]$ to sort but $a_i \in [0..d-1]$ with a small d

- I only want to sort the pairs based on a_i values ? How do I adapt my counting sort ?

```
public static void countingSort(int[] values) {
    int[] buckets = new int[100]; // k = 100
    // Count occurrences of each value in the input array
    for (int i : values) {
        buckets[i]++;
    }
    int idx = 0;
    for (int i = 0; i < 100; i++) {
        for (int j = 0; j < buckets[i]; j++) {
            values[idx] = i;
            idx++;
        }
    }
}
```

This does not work, because I'm not able to regenerate the corresponding b_i values

More Complex Counting Sort

- Suppose I have pairs $(a_i, b_i) \forall i \in [0..n-1]$ to sort based on criterion a_i only. The set of different values is small: $a_i \in [0..d-1]$ with a small d

```java
public static void countingSort(int[][] arr) {
    int n = arr.length;
    int range = 5;  // Since a_i can take values from 0 to 4

    // Step 1: Create a count array to store the frequency of each a_i
    int[] count = new int[range];

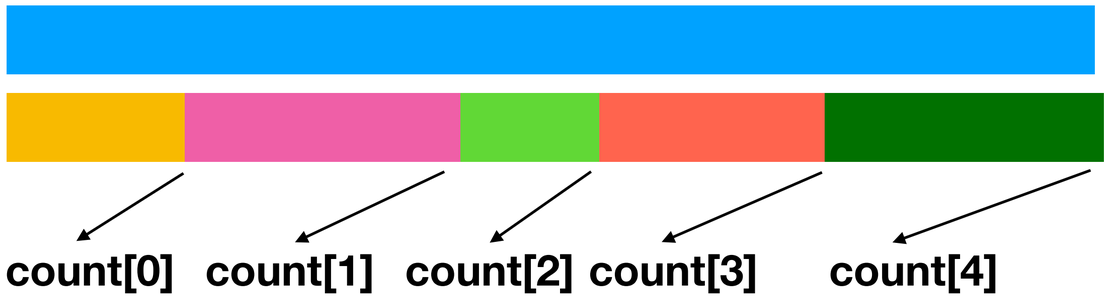
    // Step 2: Count occurrences of each a_i
    for (int i = 0; i < n; i++) {
        count[arr[i][0]]++;
    }

    // Step 3: Modify count array to store cumulative sums
    for (int i = 1; i < range; i++) {
        count[i] += count[i - 1];
    }

    // Step 4: Create an output array to store sorted pairs
    int[][] output = new int[n][2];

    // Step 5: Place the elements in the sorted order in the output array
    // Traverse the input array from right to left to maintain stable sorting
    for (int i = n - 1; i >= 0; i--) {
        int a_i = arr[i][0];
        int b_i = arr[i][1];
        output[count[a_i] - 1] = new int[]{a_i, b_i};
        count[a_i]--;
    }

    // Step 6: Copy the sorted pairs back to the original array
    for (int i = 0; i < n; i++) {
        arr[i] = output[i];
    }
}
```



Counting sort with sparse values ?

- Assuming the array to sort contains values from this set: {20007,1368910,900045}.

- Can I adapt the counting sort to still get a time complexity of Θ(n) ?

Is the counting-sort time complexity correct ?

- Because in Proposition 1 pages 280-281of the reference books it is written:

Prop. 1: No comparison-based sorting algorithm can guarantee fewer than ~N lg N comparisons in the worst case.

- But counting-sort is not based on comparisons. The comparison framework is more general.

- Also couting-sort works when there are few different values to sort compared to the number of values to sort.

Find a pair of numbers from two array summing to x

Given two arrays of numbers S1 et S2, and a number x.

Find a pair (a,b) with $a \in S1, b \in S2$ such that $a+b=x$.

Find a pair of numbers from two array summing to x

```java
public static boolean findPairWithSum(int[] S1, int[] S2, int x) {
    // Sort S2 if not already sorted
    Arrays.sort(S2);

    for (int a : S1) {
        int b = x - a;
        if (binarySearch(S2, b)) {
            System.out.println("Pair found: (" + a + ", " + b + ")");
            return true;  // Pair (a, b) exists such that a + b = x
        }
    }
    // If no such pair is found
    System.out.println("No pair found.");
    return false;
}
```

```java
// Binary search method to search for 'target' in sorted array S2
public static boolean binarySearch(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            return true;
        } else if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return false;
}
```

Find a pair of numbers from two array summing to x

Algo in two steps:

1. Sort S1
2. For each value v of S2, search x-v in S1 using binary-search

- Complexity ?
  - O(n log(n))
  - Θ(n log(n))
  - Θ(n)
  - O(n)
  - Θ(n^2)

Find a pair of numbers from a single array summing to x

- Exemple [5,10,1,150,151,155,**18**,**50**,30] x=68

- What if the array is already sorted ?
  - [1,5,10,**18**,30,**50**,150,151,155] x=68

Find a pair of numbers from a sorted array summing to x

```java
/**
 * Find a pair (i, j) in array T such that T[i] + T[j] = x.
 *
 * @param T An array of integers, sorted in non-decreasing order.
 * @param x The target sum.
 * @return An array of two integers i and j if a pair is found, or null otherwise.
 */
public static int[] findPair(int[] T, int x) {
    int i = 0;
    int j = T.length - 1;

    while (i < j) {
        if (T[i] + T[j] > x) {
            j--;
        } else if (T[i] + T[j] < x) {
            i++;
        } else {
            return new int[] {i, j};  // Pair found
        }
    }

    return null;  // Pair not found
}
```

Complexité ?

Why does it work ?

- Induction proof …



- Inductive Hypothesis: (n'importe quelle paire avec un element avant (<) i ou après (>) j ne fonctionne pas)

- Inductive Hypothesis: Any pair containing an element before (<) i or after (>) j does not work.

- Algorithm: $T[I]+T[j]>x \Longrightarrow j=j-1$ (similar for the other case)
  - We need to show that no valid pair can be formed with T[j].
  - T[j] cannot be paired with any element after i because the array is sorted, which would only yield a sum even further above x.
  - By the inductive hypothesis, T[j] cannot be paired with any element before i either.

Union of two sets (using sort)

- Two sets (represented with arrays) A et B, we must compute their union (no duplicate thus)
- A = [10,20,15,12,30], B = [10,15,20,35,13]
- A union B = [10,30,35,12,20,15,13]

Union of two sets (using sort): Solution 1

1. Copy everything in a large array

2. Sort the array

3. Remove duplicates.

- Complexity ? O((m+n) log(m+n)) where m and n are the lengths of the two arrays

Union of two sets (using sort): Solution 1

```java
public static int[] union(int[] arr1, int[] arr2) {
    int n1 = arr1.length;
    int n2 = arr2.length;

    // Step 1: Concatenate the arrays
    int[] combined = new int[n1 + n2];
    System.arraycopy(arr1, 0, combined, 0, n1);
    System.arraycopy(arr2, 0, combined, n1, n2);

    // Step 2: Sort the array
    Arrays.sort(combined);

    // Step 3: Remove duplicates
    int[] temp = new int[n1 + n2];
    int j = 0;

    for (int i = 0; i < n1 + n2 - 1; i++) {
        if (combined[i] != combined[i + 1]) {
            temp[j++] = combined[i];
        }
    }
    temp[j++] = combined[n1 + n2 - 1]; // Adding the last element

    // Trim temp array to actual size
    int[] result = new int[j];
    System.arraycopy(temp, 0, result, 0, j);

    return result;
}
```

Union of two sets (using sort): Solution 2 = merge sort-like

- Sort each array separately
- Collect elements separately avoiding duplicates.
- Complexity ? O(m log(m) + n log(n)).
  - If m = n, Solution1 has same time complexity as Solution2 using bigO

Union of two sets (using sort): Solution 2

```java
public static int[] union(int[] arr1, int[] arr2) {
    Arrays.sort(arr1);
    Arrays.sort(arr2);
    int[] temp = new int[arr1.length + arr2.length];
    int i = 0, j = 0, k = 0;
    while (i < arr1.length && j < arr2.length) {
        // Skip duplicates in arr1
        while (i < arr1.length - 1 && arr1[i] == arr1[i + 1]) i++;
        // Skip duplicates in arr2
        while (j < arr2.length - 1 && arr2[j] == arr2[j + 1]) j++;
        if (arr1[i] < arr2[j]) {
            temp[k++] = arr1[i++];
        } else if (arr1[i] > arr2[j]) {
            temp[k++] = arr2[j++];
        } else {
            temp[k++] = arr1[i];
            i++;
            j++;
        }
    }
    while (i < arr1.length) {
        if (i < arr1.length - 1 && arr1[i] == arr1[i + 1]) {
            i++;
            continue;
        }
        temp[k++] = arr1[i++];
    }
    while (j < arr2.length) {
        if (j < arr2.length - 1 && arr2[j] == arr2[j + 1]) {
            j++;
            continue;
        }
        temp[k++] = arr2[j++];
    }
    int[] result = new int[k];
    System.arraycopy(temp, 0, result, 0, k);

    return result;
}
```

Union of two sets (using sort): Solution 3

- Sort only one array,
- For each value in the second array, do a binary search on the sorted one to see if you need to add it or not.
- Which one should we sort ?
- Assume very big difference in size n (small) et n^k (large)
  - Sort the large strategy: $O(kn^k \log(n))$
  - Sort the small strategy $O(n^k \log(n))$.
  - Conclusion: The “sort the small” strategy is better by a factor k

Union of two sets (using sort): Solution 3

```java
public static int[] union(int[] arr1, int[] arr2) {
    Arrays.sort(arr1);

    int[] temp = new int[arr1.length + arr2.length];
    int k = 0, i;

    // Add unique elements from arr1 to result
    for (i = 0; i < arr1.length; i++) {
        if (i == 0 || arr1[i] != arr1[i - 1]) {
            temp[k++] = arr1[i];
        }
    }

    // Check arr2 elements against arr1
    for (int val : arr2) {
        if (!binarySearch(arr1, val)) {
            temp[k++] = val;
        }
    }

    int[] result = new int[k];
    System.arraycopy(temp, 0, result, 0, k);

    return result;
```

```java
public static boolean binarySearch(int[] arr, int target) {
    int left = 0;
    int right = arr.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) return true;
        if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }

    return false;
}
```

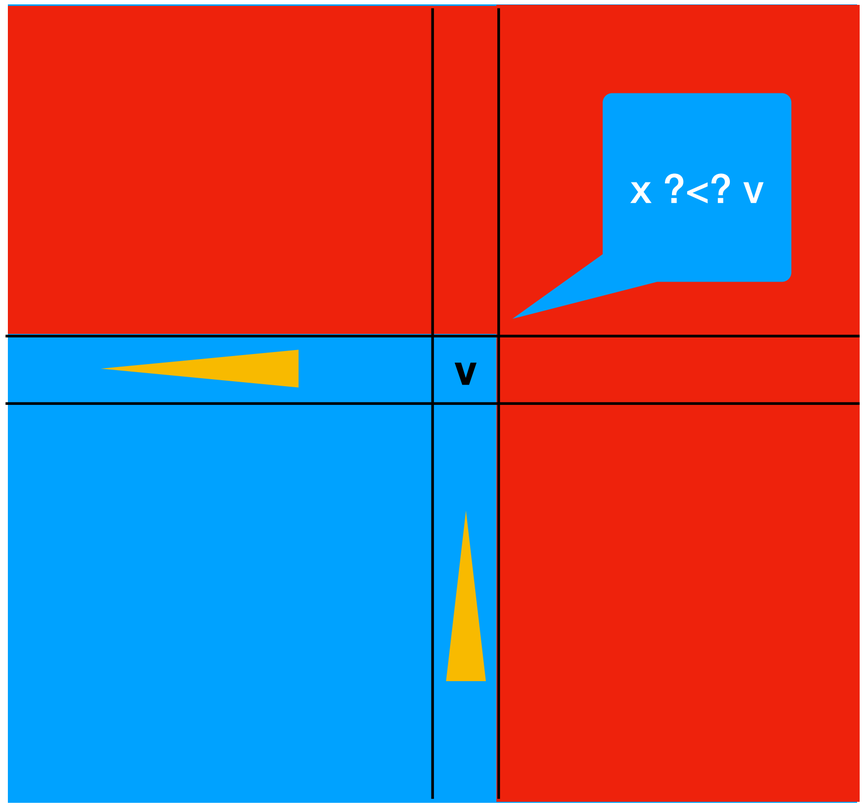
Search for a Target Value in a Row- and Column-Sorted 2D Matrix

- Given a matrix of integers that are sorted along both rows and columns, how can you efficiently find a given number in the matrix?

- Hint: There is an $O(n + m)$ time algorithm for an $n \times m$ matrix.

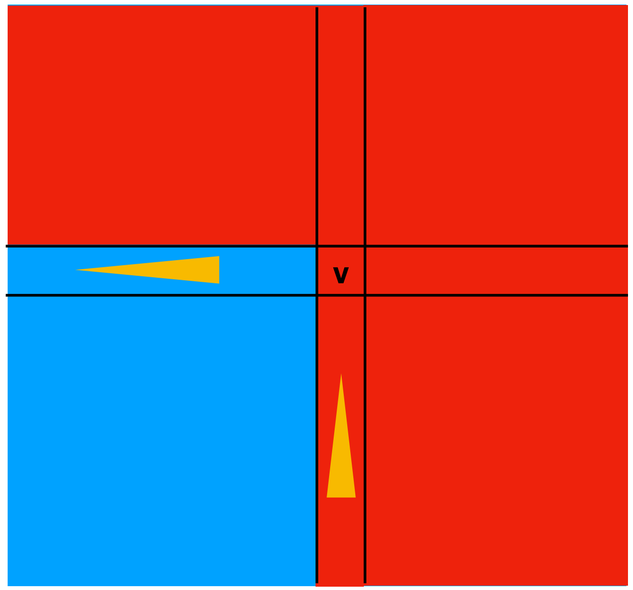
Search for a Target Value in a Row- and Column-Sorted 2D Matrix

- Start in the top-right corner and compare with the target number x.
- Which parts of the matrix can you prune from your search based on the result?

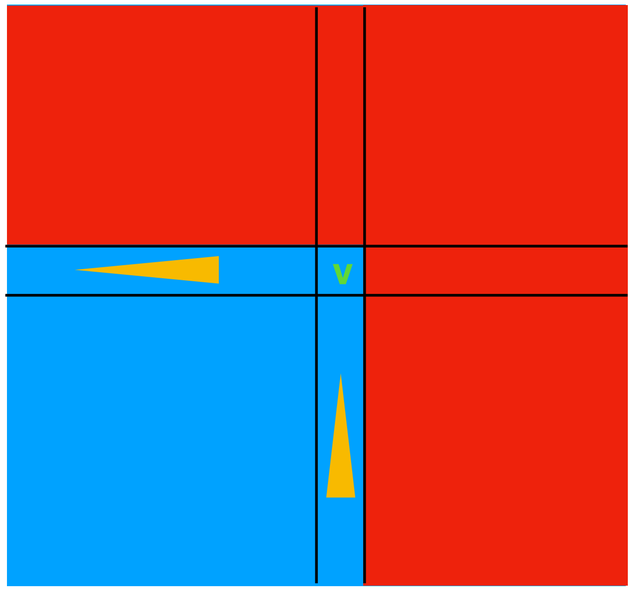


Search for a Target Value in a Row- and Column-Sorted 2D Matrix

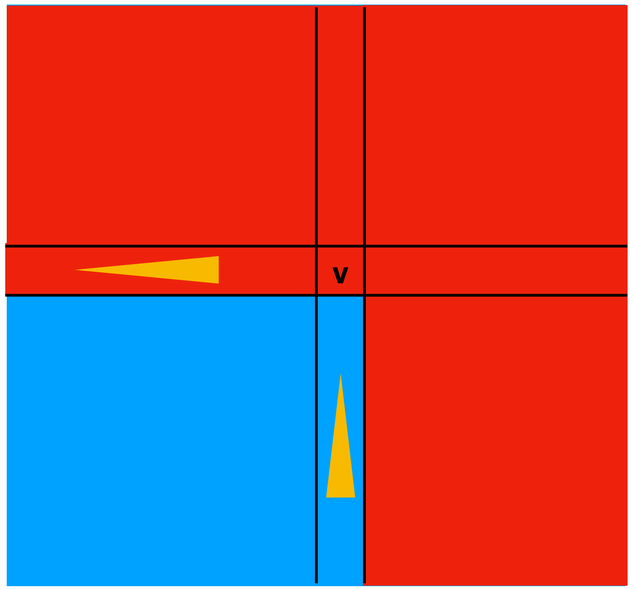
x > v



x = v

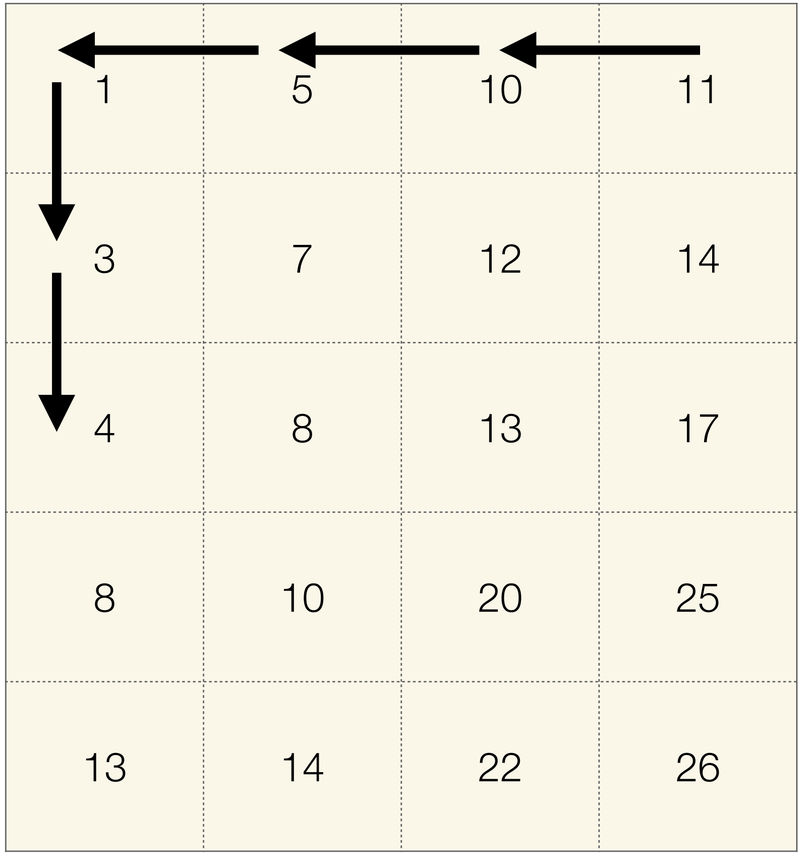


x < v

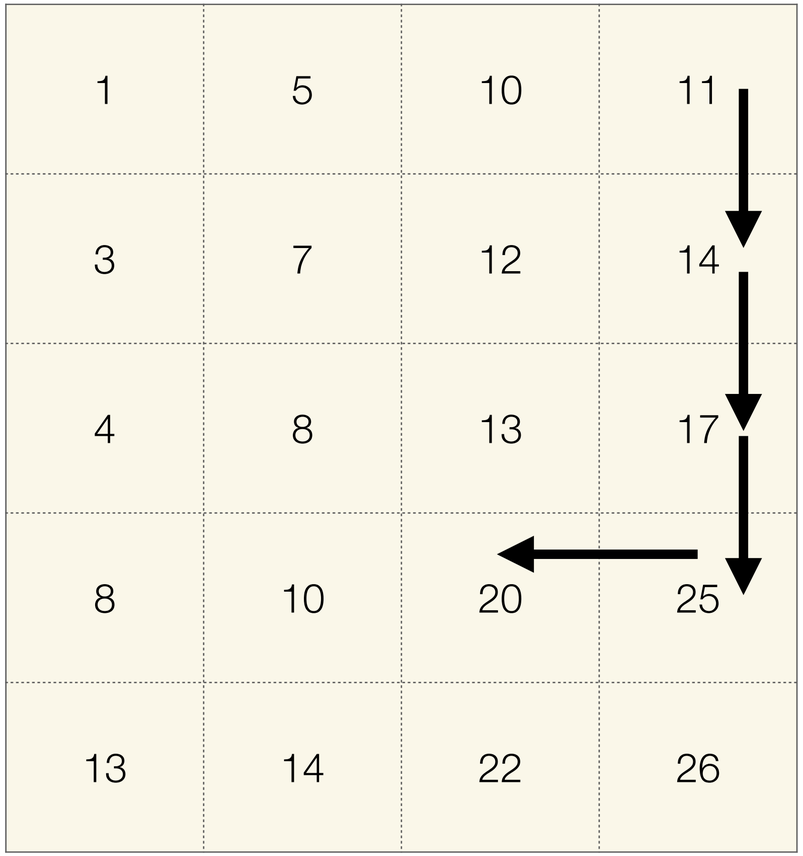


Search for a Target Value in a Row- and Column-Sorted 2D Matrix

- Recherche de 4



- Recherche de 20

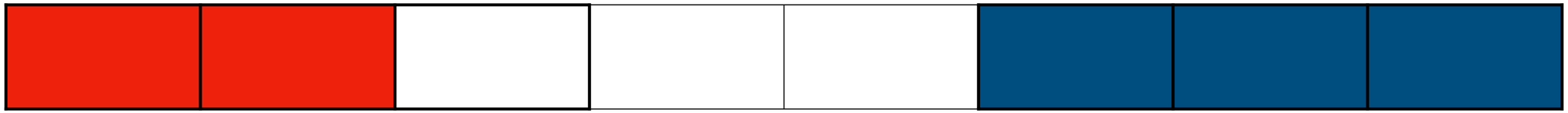


Dutch Flag Problem 🇳🇱 (proposed by Dijkstra)

Input



Output



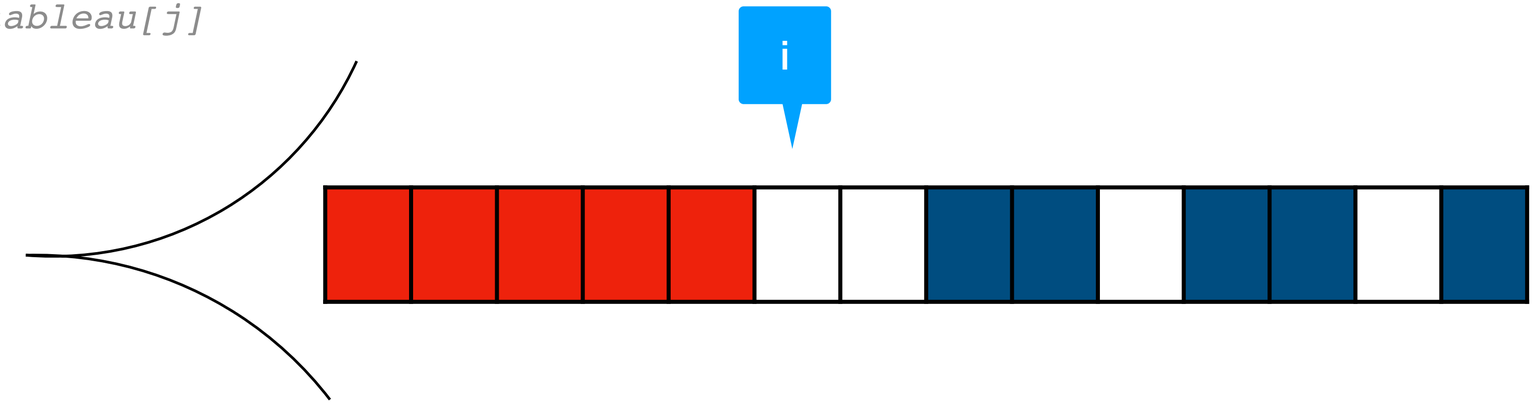
The algorithm must be “in place” (cannot create a new array or use an other one)
Discuss if the algorithm is stable or not ?

Dutch Flag: Solution 1: Quicksort partition 2 times

```
public static void flagSort(Couleur[] tableau) {
    int i = 0;

    // Partition pour ROUGE
    for (int j = 0; j < tableau.length; j++) {
        if (tableau[j] == Couleur.ROUGE) {
            // échanger tableau[i] et tableau[j]
            Couleur temp = tableau[i];
            tableau[i] = tableau[j];
            tableau[j] = temp;
            i++;
        }
    }

    // Partition pour BLANC
    for (int j = i; j < tableau.length; j++) {
        if (tableau[j] == Couleur.BLANC) {
            // échanger tableau[i] et tableau[j]
            Couleur temp = tableau[i];
            tableau[i] = tableau[j];
            tableau[j] = temp;
            i++;
        }
    }
}
```



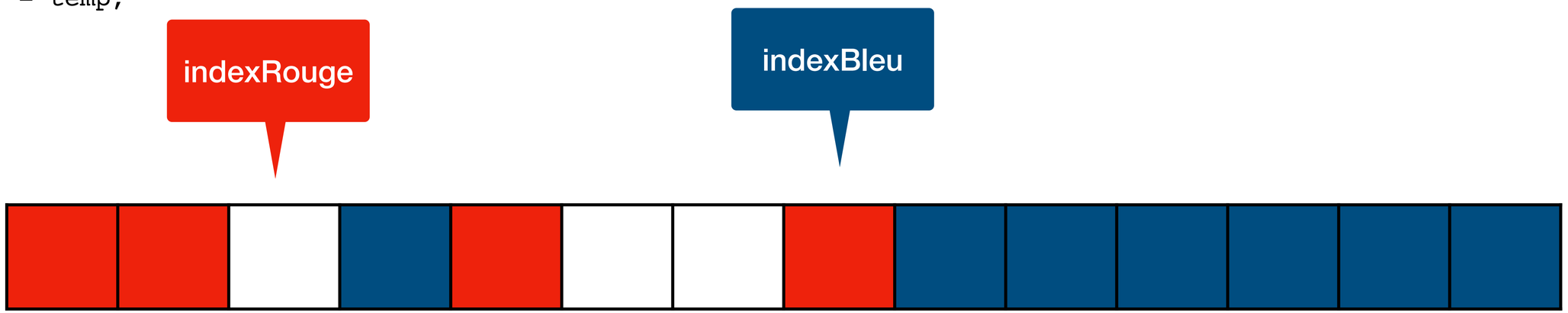
In place OK, stable for red and white but not for blue

Dutch Flag 🇳🇱: Solution 2

```java
public static void flag(Couleur[] tableau) {
    int n = tableau.length;
    int indexRouge = 0;  // Position to insert the next ROUGE color
    int indexBleu = n - 1; // Position to insert the next BLEU color
    int i = 0;
    while (i <= indexBleu) {
        if (tableau[i] == Couleur.ROUGE && i > indexRouge) {
            swap(tableau, i, indexRouge);
            indexRouge++;
        } else if (tableau[i] == Couleur.BLEU && i < indexBleu) {
            swap(tableau, i, indexBleu);
            indexBleu--;
        } else {
            i++;
        }
    }
}

public static void swap(Couleur[] tableau, int i, int j) {
    Couleur temp = tableau[i];
    tableau[i] = tableau[j];
    tableau[j] = temp;
}
```

In place OK, stable KO



Mode of an array

The mode is the value that is the most frequent in the array

Example: mode([**4**,6,2,**4**,3,1]) = 4.

- Can you adapt a well-known sorting algorithm to compute the mode of an array of n numbers ?

- What algorithm under assumption that the array only contains values from 0 to k ?

Mode of an array: Solution 1 (Two steps)

1. Sort the array

2. Find the element that repeats consecutively the most often in the sorted array

- Complexity ?
  - O(n log(n))
  - Θ(n log(n))
  - Θ(n)
  - O(n)
  - Θ(n^2)

Mode of an array: 💡instrument QuickSort

- When doing a pivot operation (*partition* method), we can count the number of elements equal to the pivot. This is a candidate for being the mode, that can replace the current best mode if more frequent.

- We can use it to avoid recursive calls in the QuicksSort.
  - Recursive call only if sub-array to partition has a length > that the current best mode, otherwise, probably no candidate mode in it.

Mode of an array: 💡instrument QuickSort

```
public class Quick3wayMode {
    static Comparable mode;
    static int bestCount = 0;

    public static Comparable mode(Comparable[] a) {
        mode = a[0]; bestCount = 1;
        sort(a, 0, a.length - 1);
        return mode;
    }

    private static void sort(Comparable[] a, int lo, int hi) {
        if (hi - lo + 1 <= bestCount) return;

        int lt = lo, i = lo + 1, gt = hi;
        Comparable v = a[lo];

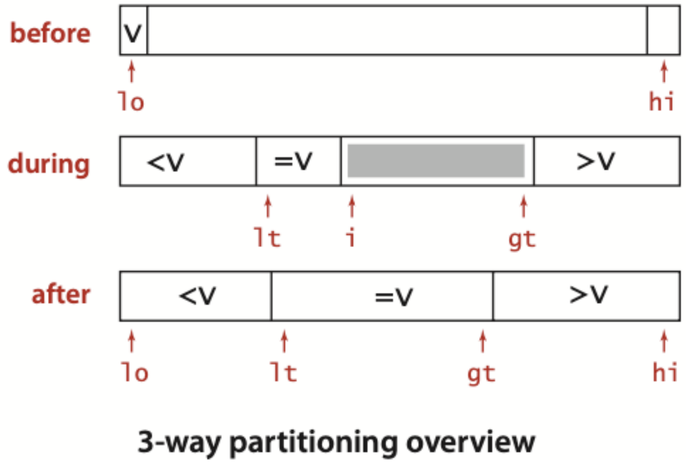
        while (i <= gt) {
            int cmp = a[i].compareTo(v);
            if      (cmp < 0) exch(a, lt++, i++);
            else if (cmp > 0) exch(a, i, gt--);
            else              i++;
        }

        if (gt - lt + 1 > bestCount) { bestCount = gt - lt + 1; mode = v; }

        if (lt - lo > bestCount) sort(a, lo, lt - 1);
        if (hi - gt > bestCount) sort(a, gt + 1, hi);    }

    private static void exch(Comparable[] a, int i, int j) {
        Comparable t = a[i]; a[i] = a[j]; a[j] = t;
    }
}
```

#v = gt-lt+1



3-way partitioning overview

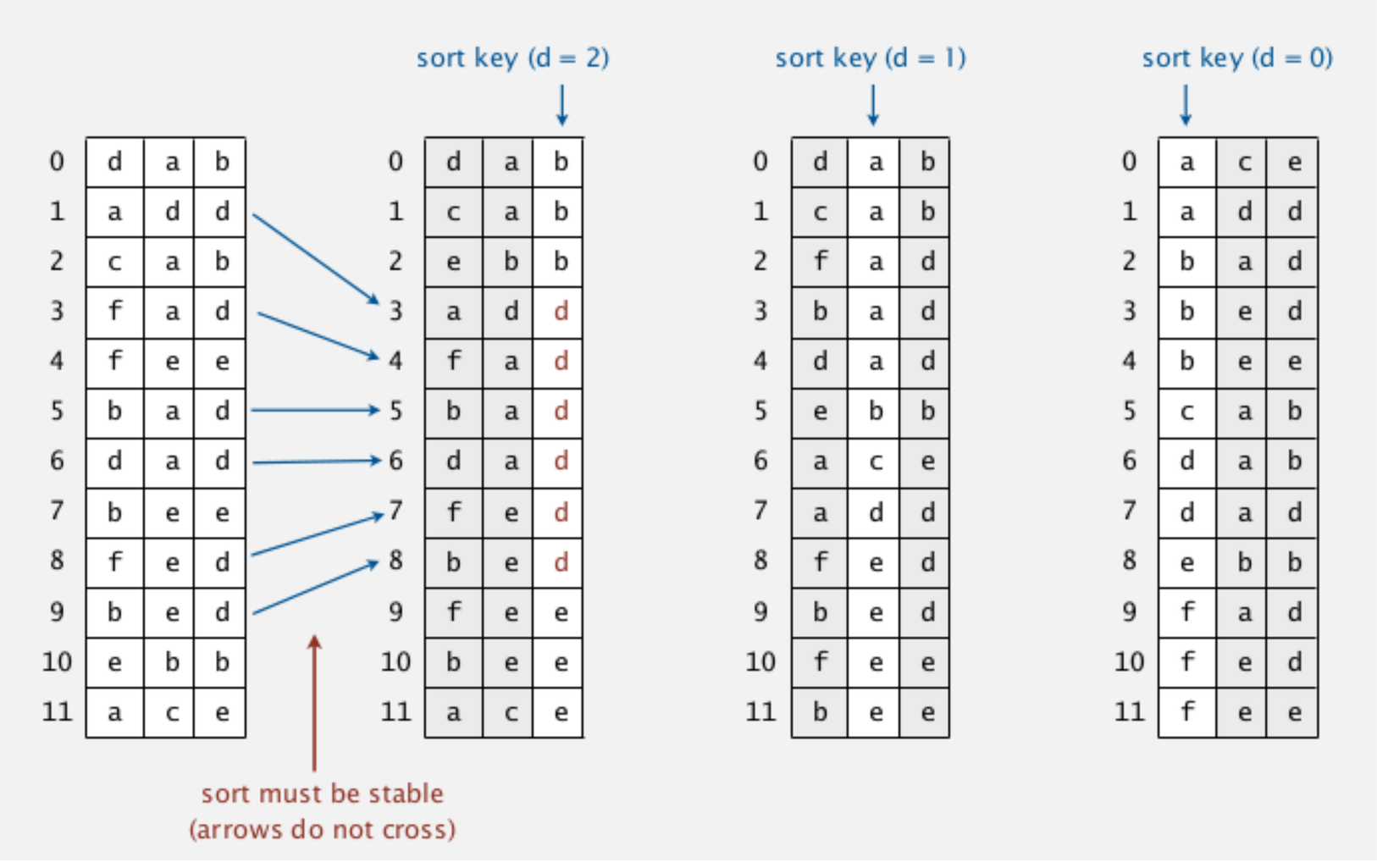
Do you recognize it ?
Same as sol2 of 🇳🇱

Do not call if lt-lo ≤ bestMode

Rappel LSD (Least Significant Digit) String Sort

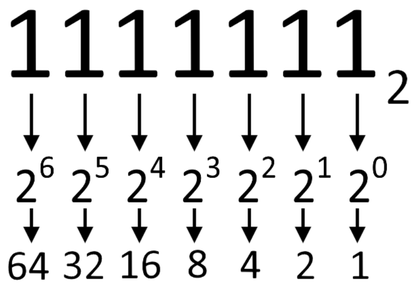
- Consider characters from right to left

- Sable-sort using the dth character as the key (using counting sort)



Binary Number = Fixed length string on alphabet of size two

- Length of int = 32 bits



- 💡Could we use LDS algorithm for sorting numbers ?

- Yes and this algorithm is called *Radix-sort*.

Preliminary on int and binary representation

- How do we know if the ith bit is 0 or 1 ?

- Like this:

```
private static int getBit(int number, int bitPosition) {
    return (number >> bitPosition) & 1;
}
```

- Example:
  - 42 (decimal) = 101010 (binary)
  - 1 (decimal) = 000001 (binary)
  - 42 >> 3 = 000101 (binary)
  - 42 >> 3 & 1 = 000001 (binary) = 1 (decimal)
-

Radix Sort = LDS Sort

```java
public static void radixSort(int[] A) {
    int maxBits = getMaxBits(A);
    int [] aux = new int[A.length];
    for (int i = 0; i < maxBits; i++) {
        stableSortOnBit(A, i, aux);
    }
}
```

```java
private static void stableSortOnBit(int[] A, int bitPosition, int[] aux) {

    int n = A.length;
    int[] count = new int[2]; // Since bits can be 0 or 1

    // Store count of occurrences in count[]
    for (int i = 0; i < n; i++) {
        int bit = getBit(A[i], bitPosition);
        count[bit]++;
    }

    count[1] += count[0];
    for (int i = n - 1; i >= 0; i--) {
        int bit = getBit(A[i], bitPosition);
        count[bit]--;
        aux[count[bit]] = A[i];
    }

    System.arraycopy(aux, 0, A, 0, n);

}
```

What is the time-complexity of radix-sort ?

Be careful, with negative numbers

- Negative number are represented in complement two

- Assuming 4 bit long numbers:
  - -5: Steps: 0101 -> (invert) -> 1010 -> (add 1) -> 1011
  - -3: Steps: 0011 -> (invert) -> 1100 -> (add 1) -> 1101
  - -2: Steps: 0010 -> (invert) -> 1101 -> (add 1) -> 1110

- Conclusion: negative numbers always start with 1, therefore if you have both positive and negative numbers, the left most bit must be sorted decreasingly.

Aggregate (example on first column)

- On first column

1	4	6
2	1	4
2	2	4
1	4	7
3	3	5
1	5	7

1	4	6
1	4	7
1	5	7
Mode	4	7

2	1	4
2	2	4
Mode	1	4

3	3	5
Mode	3	5

Result

1	4	7
2	1	4
3	3	5

Mode Computation

- Time Complexity: O(|to-from|)

```
public static int mode(int[][] array, int from, int to, int column) {
    HashMap<Integer, Integer> occurrence = new HashMap<>();
    int maxOccurrences = -1;
    int mostFrequentValue = -1;
    for (int i = from; i <= to; i++) {
        if (occurrence.containsKey(array[i][column]))
            occurrence.put(array[i][column], occurrence.get(array[i][column])+1);
        else
            occurrence.put(array[i][column], 1);

        if (occurrence.get(array[i][column]) > maxOccurrences) {
            maxOccurrences = occurrence.get(array[i][column]);
            mostFrequentValue = array[i][column];
        }

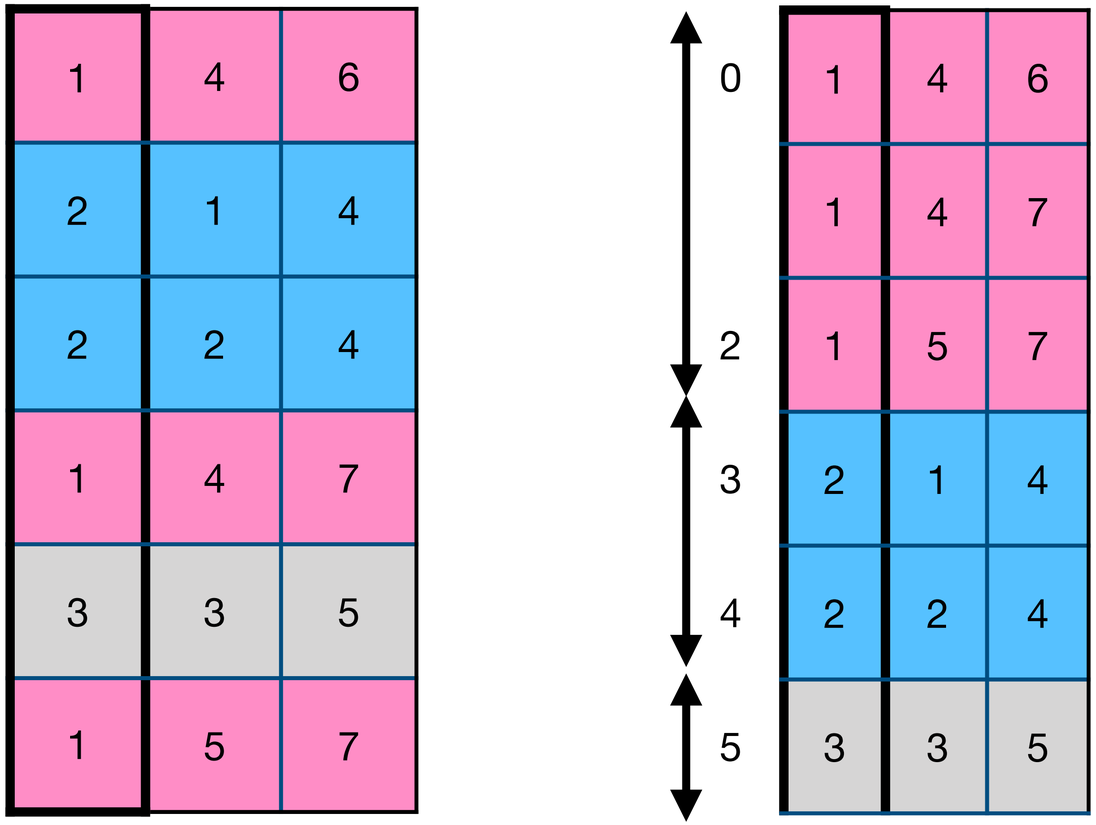
        if (occurrence.get(array[i][column]) == maxOccurrences && array[i][column] < mostFrequentValue)
            mostFrequentValue = array[i][column];
    }
    return mostFrequentValue;
}
```

Aggregate Computation

- Step 1: sort the line:

```
Arrays.sort(input, Comparator.comparingInt(a -> a[column])); //O(n.log(n))
```

Step 2: Ranges



- Step 3: compute the mode based on ranges

mode(arraySorted, 0 /*from*/,2 /*to*/, 1 /*column*/)

1	4	7
2	1	4
3	3	5



UCLouvain

LINFO1121 2026-2027

Algorithmique et Structure de données

Module 3: Search Trees

Prof pierre.schaus@uclouvain.be

Symbol tables

- ADT to
  - **Insert** one value attached to a key 🗝
  - **Retrieve** the value attached to a given key 🗝

Some examples

Application	Search Goal	Key	Value
Web search	Find relevant / interesting pages	Keywords	List of pages
Reverse DNS	Find IP address	Domain name	IP address
File system	Find a file	File name	Disk location
Compiler	Find variable properties	Variable name	Type and value
Routing table	Find a routing path for a packet	Destination	Router output interface (best route)

An array can be seen as a symbol table.

- Keys are the indices
- Values are the array entries
- Python adopts this convention. tab[i] works for both arrays (lists) and dictionaries

Reference book conventions

- A key can appear only once in the symbol table.
- Keys must not be null, and values must not be null either.
  - Indeed, get(key) returns null if the key is not present.
- Since key equality is based on the equals method, it is safer for keys to be immutable objects.

Reminder of equal method

- Must satisfy those four properties
  - Reflexivity: x.equals(x) is true
  - Symmetry: x.equals(y) $\iff$ y.equals(x)
  - Transitivity: x.equals(y) and y.equals(z) $\implies$ x.equals(z)
  - Non-null: x.equals(null) is false (or: x.equals(y) is false if y is null)

Example

```
public final class Date implements Comparable<Date>
{
    private final int month;
    private final int day;
    private final int year;

    public boolean equals(Object y)
    {
        if (y == this) return true;
        if (y == null) return false;
        if (y.getClass() != this.getClass())
            return false;
        Date that = (Date) y;
        if (this.day   != that.day  ) return false;
        if (this.month != that.month) return false;
        if (this.year  != that.year ) return false;
        return true;
    }
}
```

Base contract

The class is final, not question to ask

cast cannot fail

Test all the fields (primitive types here), be careful == vs equals

Implémentation possible des tables de symboles

- Implémentations possibles pour une tables de symboles:
  - Sequential Search = simple liste chainée qui contient les clefs et les valeurs
    - insert(key,value) en O(n),
    - get(key)/delete(key) en O(n)

- En partie 4 nous verrons une autre structure (table de hashage) qui permet O(1) amortie pour toutes les opérations. Patience …

Order between keys

- We can extend the ADT with new methods

public class ST<Key extends Comparable<Key>, Value>	
...	
Key min()	*smallest key*
Key max()	*largest key*
Key floor(Key key)	*largest key less than or equal to key*
Key ceiling(Key key)	*smallest key greater than or equal to key*
int rank(Key key)	*number of keys less than key*
Key select(int k)	*key of rank k*
void deleteMin()	*delete smallest key*
void deleteMax()	*delete largest key*
int size(Key lo, Key hi)	*number of keys between lo and hi*
Iterable<Key> keys()	*all keys, in sorted order*
Iterable<Key> keys(Key lo, Key hi)	*keys between lo and hi, in sorted order*

Using comparable keys

- The search can be improved using binary search rather than sequential one

	sequential search	binary search
search	N	$\log N$
insert / delete	N	N
min / max	N	1
floor / ceiling	N	$\log N$
rank	N	$\log N$
select	N	1
ordered iteration	$N \log N$	N

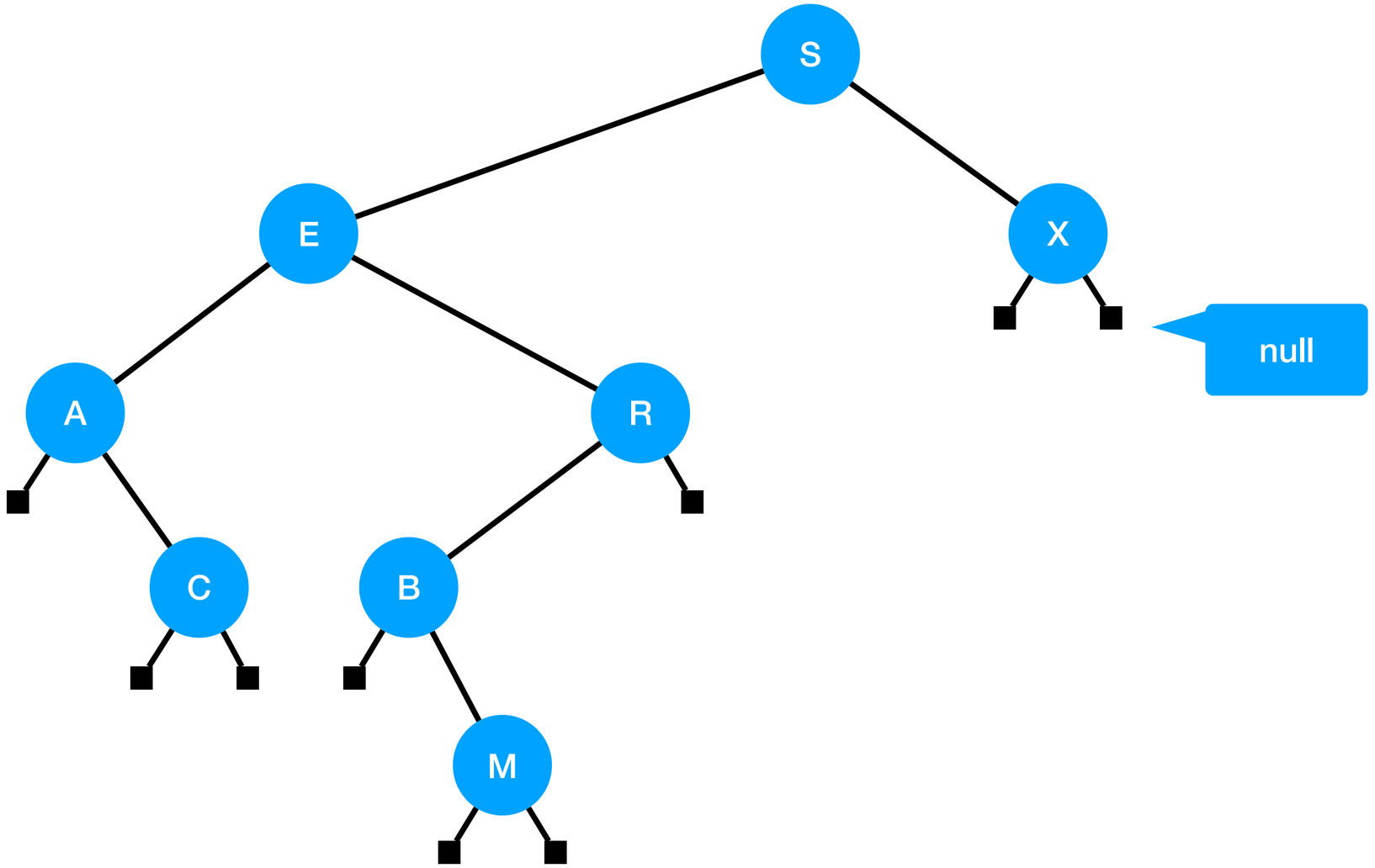
Can be improved using a new data-structure called Binary Search Tree (BST)

Les arbres de recherche

- Binary tree:
  - Empty
  - Non-empty: Contains two binary trees (left and right)
- Binary search tree = binary tree with a key-value pair in each node such that:
  - This key is strictly greater than all keys in the left subtree
  - This key is strictly less than all keys in the right subtree

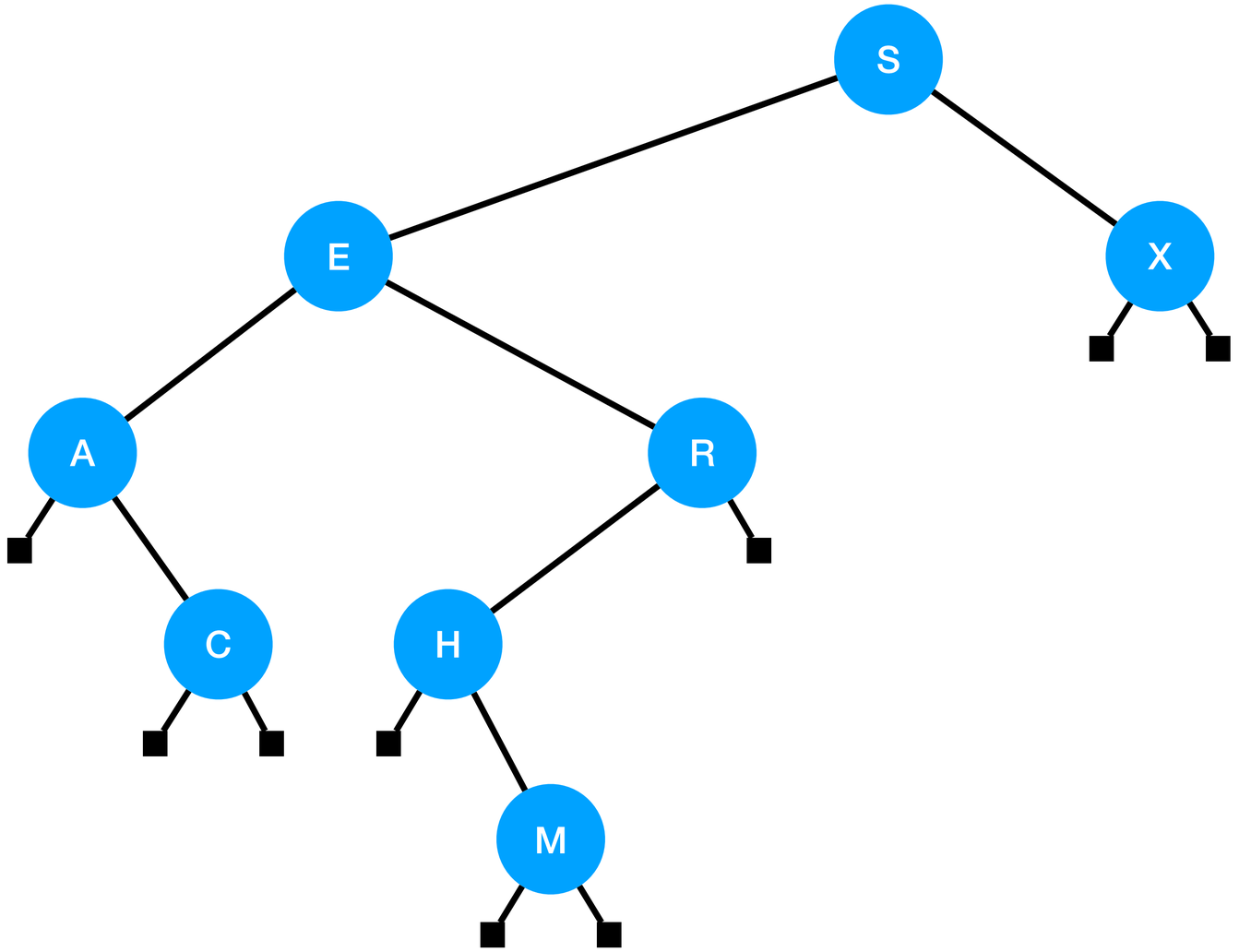
Example

- Valid BST ?



Example

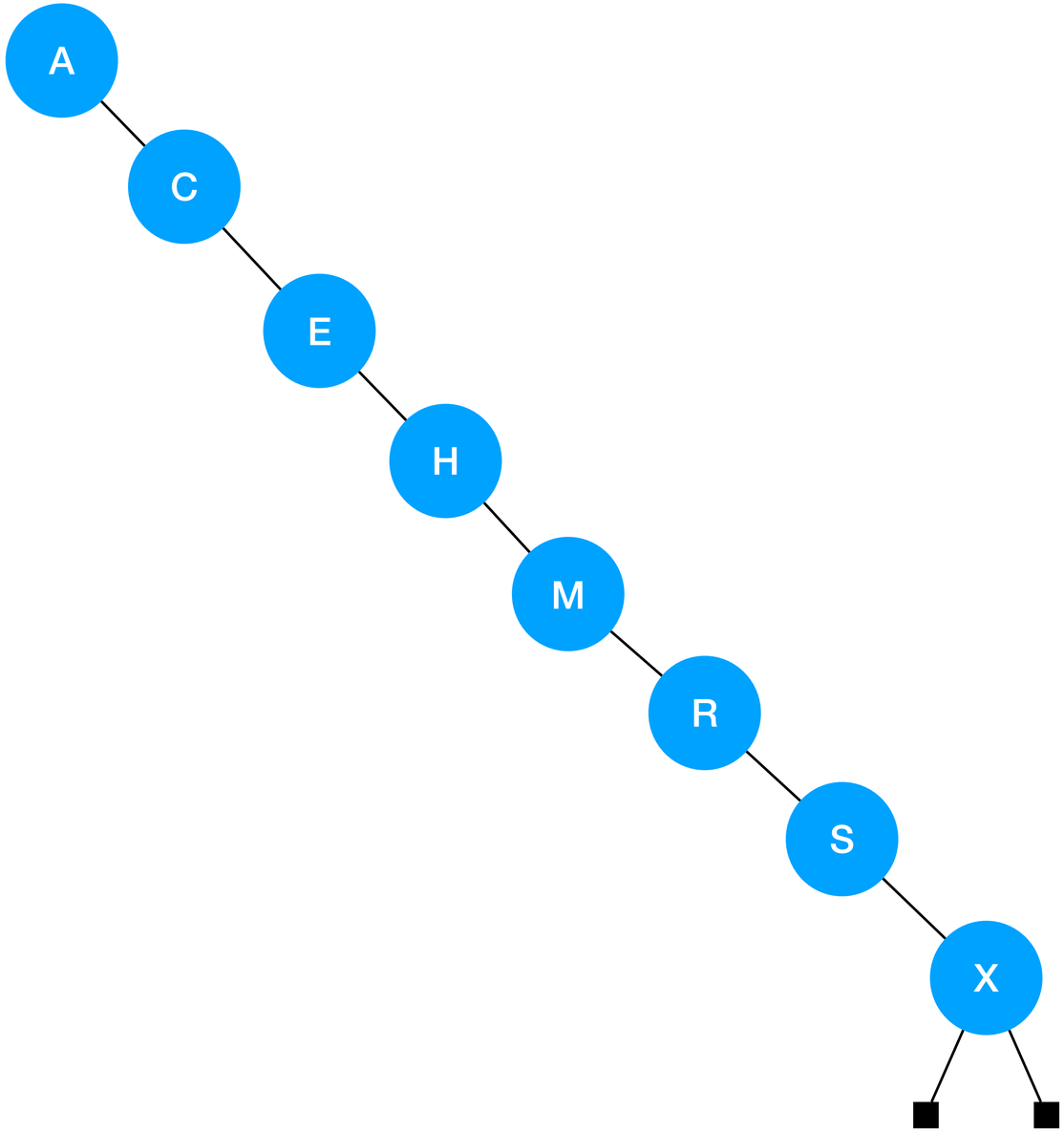
- Valid BST ?



Search for a key in a binary search tree

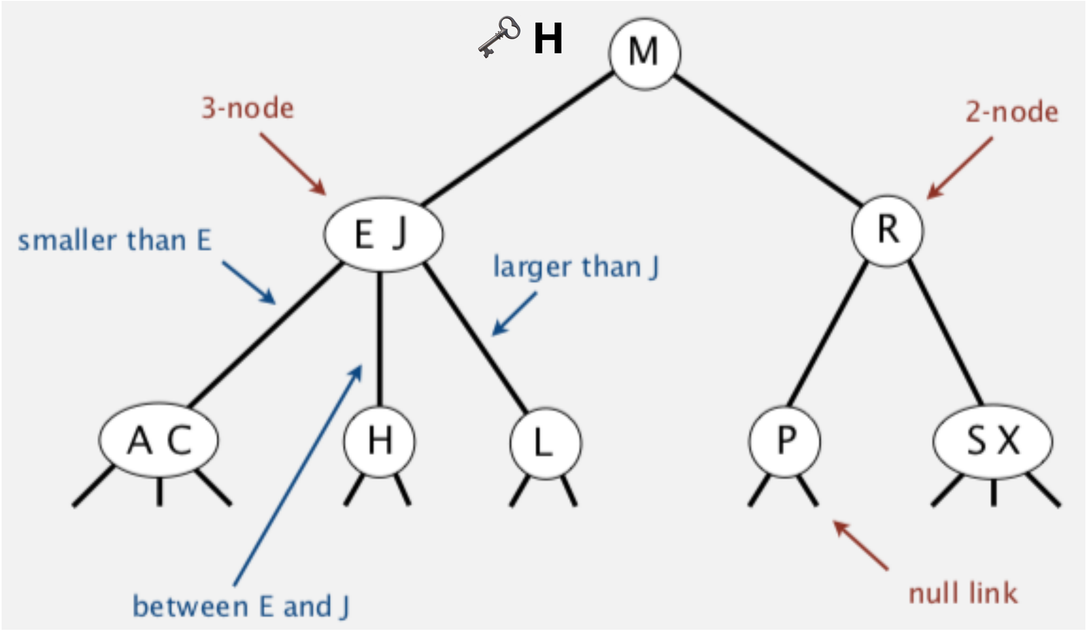
- Complexity ?
  - O(n), where n is the number of nodes
  - Θ(n), where n is the number of nodes
  - O(log n), where n is the number of nodes
  - Θ(log n), where n is the number of nodes
  - O(h), where h is the height of the tree
  - Θ(h), where h is the height of the tree

Wort case time complexity for the height in a BST ?



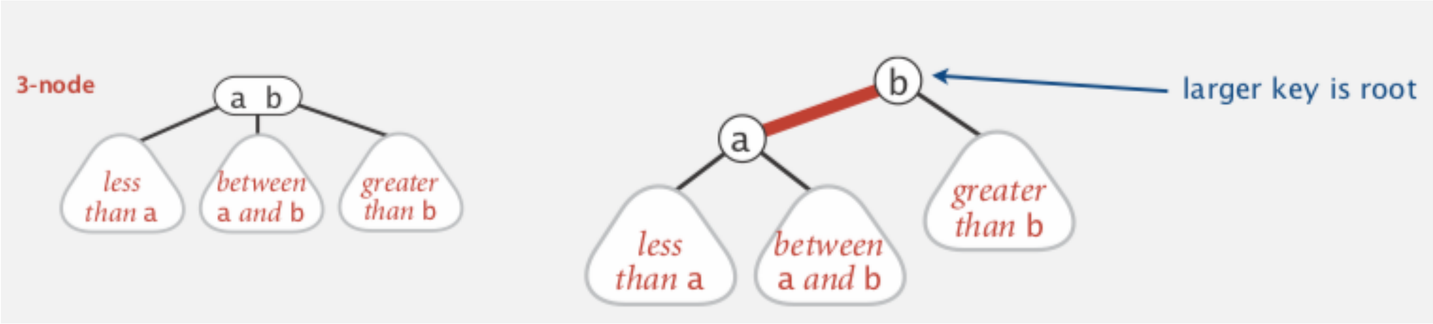
In a balanced search tree O(h) = O(log(n))

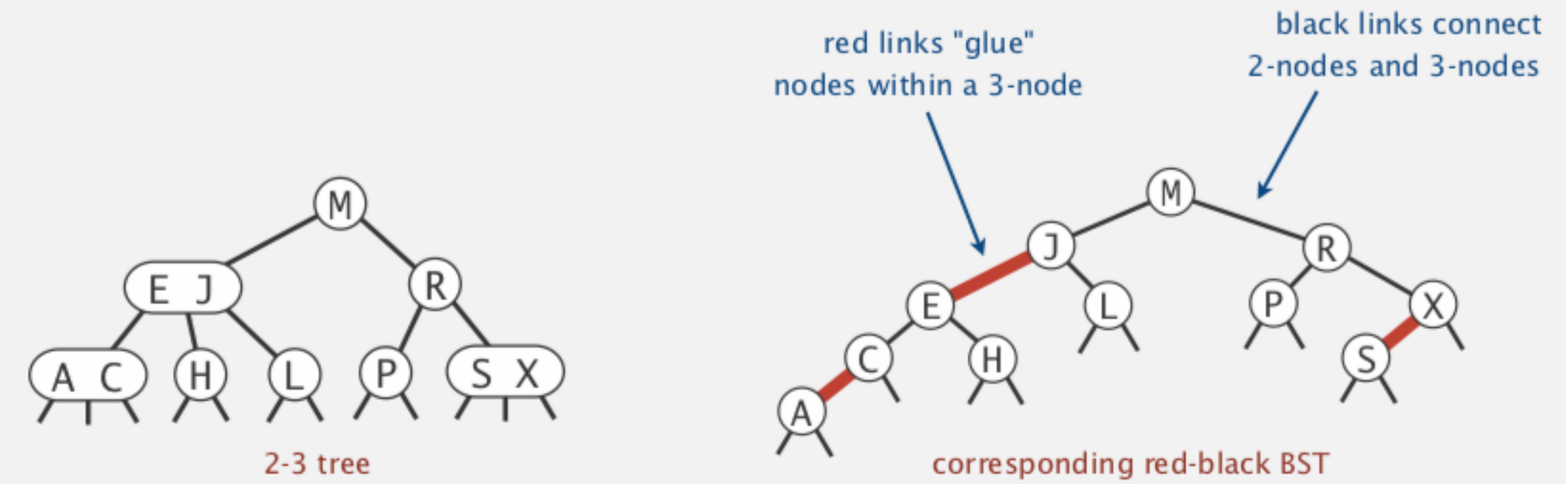
- 2-3 search trees have 1 or 2 keys per node. Properties
  - Perfect balance: Every path from the root to a leaf (null link) has exactly the same length.
  - The advantage is that complexities are guaranteed to be O(log n). The "linked list" effect is no longer possible.



Balanced Binary Search Tree

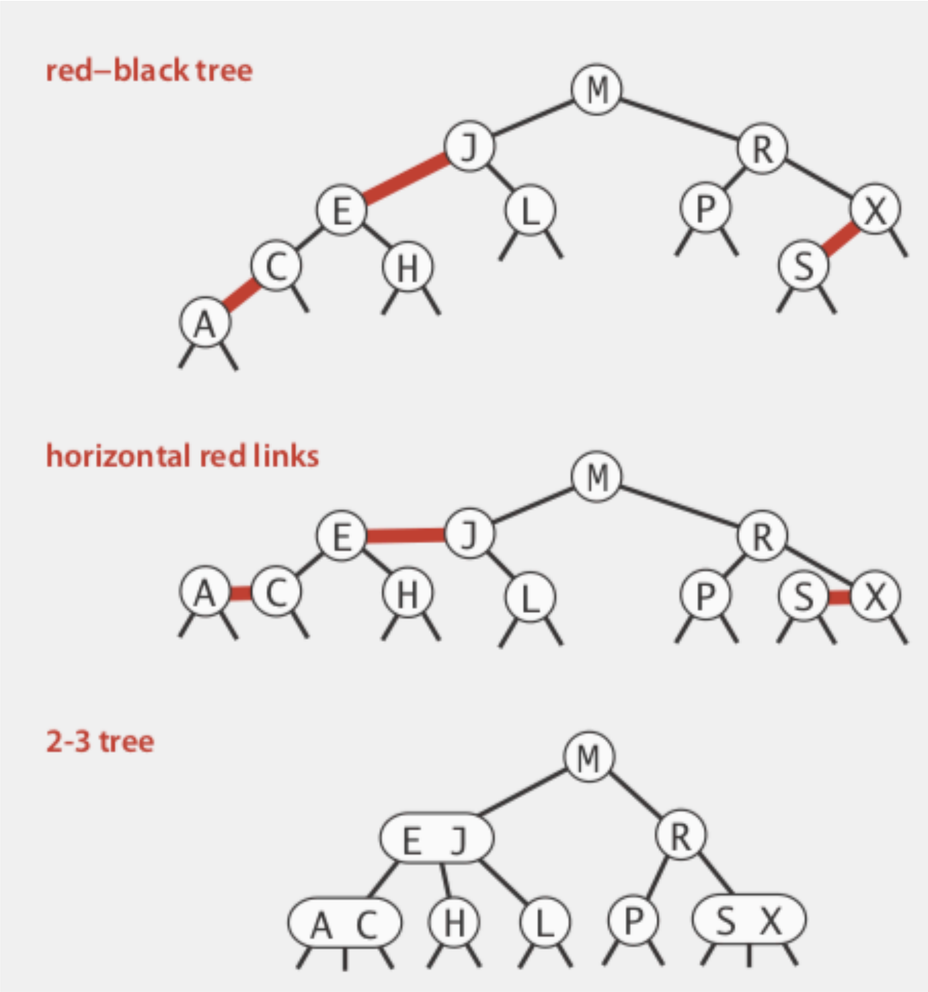
- Left leaning red-black tree = a 2-3 search tree encoded with a binary tree.





Red-black vs 2-3

- One to one mapping
  - To realize it, put the left links horizontally



Summary of time complexity

implementation	guarantee		
	search	insert	delete
sequential search (unordered list)	N	N	N
binary search (ordered array)	$\lg N$	N	N
BST	N	N	N
2-3 tree	$c \lg N$	$c \lg N$	$c \lg N$
red-black BST	$2 \lg N$	$2 \lg N$	$2 \lg N$

Next two weeks

- In-depth reading of Chapters 3.1, 3.2, and 3.3
- Theoretical exercises to master search trees and balanced search trees
- Implementation exercises (parts 1–3)
- A quiz will be offered in Week 7 (a quite difficult one) to prepare for the exam. It may take more than 2 hours.